

Kapitel 25. Namespaces

Innehållsförteckning

[Vad är ett *namespace*](#)

[Använda namespaces](#)

[Skapa egna namespaces](#)

[Alias](#)

[Diskussion](#)

Detta kapitel behandlar hur man kan gruppera kod i logiska "namnrymder" för att undvika att klasser och funktioner med samma namn skall orsaka problem.

Vad är ett *namespace*

Namespaces används i C++ för att gruppera kod i logiska helheter. De fungerar på en högre nivå än klasser och funktioner, och används för att just gruppera klasser, funktioner och andra definitioner i en gemensam namnrymd. Genom att gruppera kod i namespaces kan man undvika problem som kan framkomma i stora projekt, nämligen att t.ex. samma klassnamn syftar till två eller flera helt olika klasser. Man får då problem med kompileringen av projektet. Med hjälp av namespaces kan man gruppera olika delar av ett stort projekt i olika namespaces och därmed undvika kollisioner.

Exempel på namnkollision

Vi kan titta på ett exempel på ett program där en namnkollision förekommer. Vi antar att vi har ett program som skall visualisera grafik i 3D. Vi använder i detta program den klass för att representera vektorer som presenterades i [Kapitel 21](#). Vi kan dock för vårt exemplars skull anta att man kallat klassen `vector` istället för `Vector`. Programmet använder sig även av normala vektor-containers från STL (se [Kapitel 24](#)) för att lagra våra matematiska vektorer. Vad händer om vi försöker göra kod enligt följande:

```
#include <vector>

class vector {
public:
    vector () {} ;
};

int main () {
    // skapa en vektor av vektorer
    vector<vector> VektorContainer;
}
```

Exemplet ovan är en aning överdrivet, men det kommer inte att kunna kompileras. Kompilatorn kommer att klaga på något i stil med att `vector` definieras om på rad 3. Vi kan således inte ha två olika klasser med samma namn och använda dem i samma fil. De enda lösningen på problemet med våra kunskaper hittills är att döpa om vår egna matematiska `vector`-klass till något annat som inte kolliderar med klassen från STL.

Vad skulle ha hänt ifall matematik-`vector` skulle komma från ett bibliotek till vilket vi inte har

tillgång till källkoden och kan ändra på klassens namn? I så fall har vi två klasser `vector` och vi kan inte ändra på namnet på någon av dem. Lösningen här är att utnyttja namespaces.

Namespacet `std`

Det mesta av de bibliotek som tillhör C++ finns logiskt organiserat i ett enda namespace som heter `std` (förkortning av `standard`). Detta gäller alla klasser som har att göra med t.ex. input/output och STL. På så sätt finns alla standard-klasser och definitioner under "ett tak" och man kan undvika kollisioner med egna eller tredje parts klasser och definitioner.

[Förutgående](#)

Diskussion

[Hem](#)

[Nästa](#)

Använda namespaces